E S C U E L A P O L I T É C N I C A N A C I O N A L

Facultad de Ingeniería de Sistemas · Carrera de Ciencia de Datos e Inteligencia Artificial

INFORME DE LABORATORIO · SEMANA 3

Clasificación de variables y población vs. muestra

Laboratorio complementario · Semana 3

Asignatura IDSD422 — Estadística y Programación para Ciencia de Datos I

Estudiante(s)	Guanocunga Ojeda, Alejandro Mateo — usuario de GitHub: alejandro-guanocunga
Docente	Christian Suárez
Período académico	2026-B
Fecha de entrega	23/10/2026
Repositorio (fork)	https://github.com/alejandro-guanocunga/IDSD422-lab-muestreo-poblacion

1 · Introducción

La empresa RetailAndina S.A. enfrenta la necesidad estratégica de auditar su base de clientes de fidelización antes de emprender iniciativas analíticas o modelos predictivos de comportamiento de consumo. En el ámbito de la ciencia de datos y la inteligencia de negocios, operar con atributos incorrectamente categorizados o ignorar la naturaleza de las variables distorsiona los cálculos agregados e induce a conclusiones inválidas ante la dirección ejecutiva. Asimismo, estimar magnitudes económicas críticas a partir de subconjuntos reducidos requiere constatar formalmente que la muestra represente la estructura intrínseca de la población. Una muestra mal recolectada introduce sesgos sistemáticos que ningún aumento posterior en el número de observaciones consigue subsanar.

Como analista de BI, asumí la responsabilidad de examinar la base piloto data/clientes.csv (12 clientes, 7 variables analíticas) y contrastar la precisión inferencial de tres metodologías de muestreo aplicadas sobre la población completa data/clientes_400.csv (N = 400). Demostré en código reproducible que el diseño de muestreo decide la validez de los resultados, no el tamaño de la muestra.

2 · Objetivos

Objetivo general

Clasificar las variables de la base de fidelización de RetailAndina S.A. aplicando criterios formales de medición y contrastar la representatividad y precisión de tres diseños de muestreo frente al parámetro poblacional mediante flujos analíticos reproducibles en Python y R.

Objetivos específicos

1. Clasificar formalmente las 7 variables de data/clientes.csv según su escala estadística (nominal, ordinal, discreta o continua) y contrastarla con la inferencia de tipos de datos (dtypes) y valores únicos en código.

2. Declarar explícitamente el orden jerárquico de las variables ordinales mediante estructuras categóricas nativas (CategoricalDtype en Python y factores ordenados en R), erradicando ordenamientos alfabéticos espurios.

3. Calcular el parámetro poblacional exacto de ingresos mensuales en data/clientes_400.csv e implementar tres estrategias de muestreo (conveniencia, aleatorio simple y estratificado proporcional) con tamaño n = 20.

4. Evaluar cuantitativamente el sesgo de selección del muestreo por conveniencia mediante la distribución por ciudad y cuantificar la variabilidad del sorteo aleatorio simple a través de 5 semillas deterministas.

3 · Desarrollo de la práctica

3.1 · Configuración del entorno y control de versiones con Git

Para garantizar la reproducibilidad y el aislamiento analítico, ejecuté la práctica en Google Colab vinculado con mi repositorio de GitHub (fork: alejandro-guanocunga/IDSD422-lab-muestreo-poblacion). Tras definir mi identidad con git config, realicé la clonación limpia y comprobé la estructura de carpetas data/, scripts/ e informes/ (Figura 1).

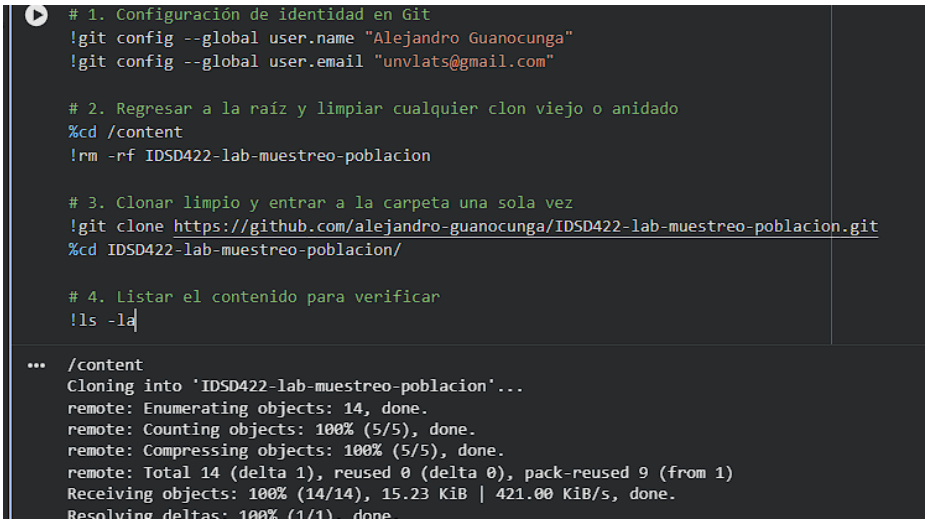


Figura 1. Configuración de credenciales de Git, clonación del repositorio y verificación del entorno en Google Colab.

3.2 · Parte A · Clasificación y verificación de variables

Analicé la base data/clientes.csv (12 clientes, 7 variables + cliente_id). Apliqué criterios formales de la teoría de la medición (Stevens, 1946; Bruce et al., 2020): continuidad matemática (divisibilidad y sentido de valores decimales), existencia de cero absoluto no arbitrario (escala de razón), jerarquía u orden intrínseco natural, y conteo discreto. Implementé scripts/clasificacion_variables.py verificando dtypes inferidos, explorando valores únicos y asignando tipo categórico ordenado (pd.CategoricalDtype) a las ordinales. Los resultados se resumen en la Tabla 1.

Tabla 1. Clasificación conceptual, justificación técnica y verificación en código de las variables de clientes.csv.

Variable	Tipo estadístico	Justificación (criterio formal)	Verificación en código (dtypes y únicos)
cliente_id	Cualitativa nominal (Identificador)	Etiqueta alfanumérica única. No expresa magnitud ni admite ordenación jerárquica u operaciones matemáticas.	Inferido object (Python) / chr (R). 12 valores únicos (C001–C012), confirmando su rol de identificador unívoco.
edad	Cuantitativa continua (discretizada en razón)	Tiempo vivido sobre un continuo temporal con cero absoluto real (tiene sentido fraccionario, ej. 27.5 años). En el negocio se capturó en años enteros cumplidos.	Inferido int64 (Python) / int (R). 12 valores únicos observados (22 a 48 años). Mantiene propiedades de razón.
numero_hijos	Cuantitativa discreta	Conteo de entidades biológicas indivisibles. No admite valores decimales (no tiene sentido 1.5 hijos) y posee un cero absoluto natural.	Inferido int64 (Python) / int (R). 5 valores únicos observados: [0, 1, 2, 3, 4].
genero	Cualitativa nominal	Categorías cualitativas mutuamente excluyentes sin jerarquía ni precedencia intrínseca natural.	Inferido object (Python) / chr (R). 3 valores únicos reportados: ['Femenino', 'Masculino', 'Otro'].
nivel_educacion	Cualitativa ordinal	Progresión formativa que guarda jerarquía estricta (Primaria < Secundaria < Universidad < Posgrado), sin distancia métrica constante.	Inferido object (4 únicos); transformado explícitamente a category ordenada mediante CategoricalDtype en Python y Ord.factor en R.
ciudad	Cualitativa nominal	Atributo geográfico que denota ubicación territorial. No admite ordenación matemática natural.	Inferido object (Python) / chr (R). 3 valores únicos identificados: ['Quito', 'Guayaquil', 'Cuenca'].
ingresos_mensuales	Cuantitativa continua	Flujo monetario en USD. Admite infinitas subdivisiones fraccionarias (centavos) y posee cero absoluto real.	Inferido int64 (Python) / int (R) debido al redondeo en la captura (12 únicos, rango $760 a $2450 USD).
satisfaccion	Cualitativa ordinal (Escala Likert)	Medición psicométrica de percepción codificada de 2 a 5. Posee jerarquía estricta pero carece de equidistancia métrica entre niveles.	Inferido int64 (Python) / int (R). Se declaró el orden explícito convirtiéndolo a category ordered en Python y factor en R.

3.3 · Pregunta 1

P1. ¿Qué te dijo el dtype de satisfaccion y por qué no basta para clasificarla? ¿Qué pasa si una ordinal queda como texto sin declarar el orden? (Responde con la evidencia de tu ejecución.)

La ejecución en consola reveló que pandas infirió satisfaccion como int64 (y R como int). El tipo de dato computacional no basta para clasificar una variable porque las computadoras asignan dtypes con base en el formato de almacenamiento en memoria y no en la semántica del constructo. En clientes.csv los valores observados son 4, 3, 5 y 2, los cuales corresponden a códigos numéricos de una escala Likert de percepción. Si un analista confía a ciegas en el dtype numérico, podría cometer el grave error de aplicar operaciones aritméticas inválidas, como promediar o calcular desviaciones estándar, asumiendo una escala de intervalo cuando en realidad no existe garantía de que la distancia perceptual entre '2' (mala) y '3' (regular) sea equivalente a la distancia entre '4' (buena) y '5' (excelente).

Por otra parte, si una variable ordinal se almacena como texto simple (object o chr) sin declarar el orden, el software aplica ordenamiento lexicográfico por defecto. En mi ejecución en Python y R comprobé que nivel_educacion se ordenaría alfabéticamente como Posgrado > Primaria > Secundaria > Universidad, distorsionando por completo la jerarquía formativa. Al declarar explícitamente pd.CategoricalDtype(categories=[2, 3, 4, 5], ordered=True) en Python y factor(levels=c('2','3','4','5'), ordered=TRUE) en R, se garantiza que cualquier gráfico, filtro o cómputo respete la relación estricta $2 < 3 < 4 < 5$.

3.4 · Parte B · Población vs. muestra: Diseños muestrales y análisis de sesgo

Examiné la población data/clientes_400.csv (N = 400 clientes: Quito 200, Guayaquil 120, Cuenca 80). La variable de interés es ingresos_mensuales. Primero calculé el parámetro poblacional exacto (μ) mediante la media sobre los 400 clientes, resultando exactamente en 1330.72 USD (Tabla 2).

Tabla 2. Población, variable de interés, parámetro y justificación del muestreo.

Población (N)	N = 400 clientes en data/clientes_400.csv (Quito: 200, Guayaquil: 120, Cuenca: 80).
Variable de interés	ingresos_mensuales (variable cuantitativa continua en dólares estadounidenses USD).
Parámetro poblacional (media μ)	μ = 1330.72 USD (calculado vectorialmente sobre los 400 clientes de la población completa).
¿Por qué se muestrea si aquí el parámetro existe y se calculó?	En el mundo real censar toda la población suele ser imposible por costos y tiempos. Aquí el parámetro conocido funge como patrón de verdad (ground truth) para contrastar objetivamente el sesgo y la varianza de cada diseño muestral.

Implementé en scripts/muestreo.py tres diseños con tamaño n = 20 (5 % de la población): conveniencia (head(20)), aleatorio simple (sample(20, random_state=1)) y estratificado proporcional por ciudad (groupby('ciudad').sample(frac=0.05, random_state=1)). Los resultados se resumen en la Tabla 3.

Tabla 3. Desempeño comparativo de los tres diseños muestrales frente al parámetro (μ = 1330.72 USD).

Diseño de muestreo	n	Media estimada	Composición por ciudad	Diferencia vs. parámetro
Conveniencia (head)	20	1348.50 USD	Quito: 20 (100 %) Guayaquil: 0 (0 %) Cuenca: 0 (0 %)	**+17.78 USD (+1.34 %)**
Aleatorio simple (sample, seed=1)	20	1336.00 USD	Quito: 8 (40 %) Cuenca: 8 (40 %) Guayaquil: 4 (20 %)	**+5.28 USD (+0.40 %)**
Estratificado (groupby + 5 %)	20	1287.50 USD	Quito: 10 (50 %) Guayaquil: 6 (30 %) Cuenca: 4 (20 %)	**-43.22 USD (-3.25 %)**

```
!python scripts/muestreo.py

Población: 400 clientes × 8 columnas
Parámetro poblacional — ingreso medio: 1330.72 USD
Ciudades en la población: {'Quito': 200, 'Guayaquil': 120, 'Cuenca': 80}

--- Medias obtenidas con semillas 1 a 5 (Aleatorio simple) ---
Semilla 1: media = 1336.00 USD
Semilla 2: media = 1405.00 USD
Semilla 3: media = 1235.00 USD
Semilla 4: media = 1491.50 USD
Semilla 5: media = 1288.00 USD

[Conveniencia] n=20  media=1348.50 USD  (parámetro 1330.72, diferencia +17.78)
    composición por ciudad: {'Quito': 20}
[Aleatorio simple] n=20  media=1336.00 USD  (parámetro 1330.72, diferencia +5.28)
    composición por ciudad: {'Cuenca': 8, 'Quito': 8, 'Guayaquil': 4}
[Estratificado] n=20  media=1287.50 USD  (parámetro 1330.72, diferencia -43.22)
    composición por ciudad: {'Quito': 10, 'Guayaquil': 6, 'Cuenca': 4}

Parámetro poblacional: 1330.72 USD
                    diseño  n  media estimada
          Conveniencia (head) 20         1348.5
     Aleatorio simple (sample) 20         1336.0
Estratificado (groupby + frac) 20         1287.5

Con estas salidas responde en informe.md las preguntas P2 a P5:
  P2: ¿cuál diseño quedó más cerca del parámetro y por qué un solo
      número no basta para declarar un ganador?
  P3: ¿qué defecto tiene la muestra por conveniencia y cómo lo delata
      su composición por ciudad? ¿Se corrige con más filas?
  P4: ¿qué implica la variabilidad entre semillas para una decisión
      tomada con un solo estudio?
  P5: ¿cuándo usarías cada diseño? (piloto · subgrupos · conclusión)
```

Figura 2. Salida en consola de scripts/muestreo.py: parámetro real (1330.72 USD), variabilidad entre semillas 1 a 5 y reporte comparativo de los tres diseños muestrales.

Tabla 4. Variabilidad muestral del aleatorio simple en 5 sorteos independientes (semillas 1 a 5).

Entorno / Métrica	Semilla 1	Semilla 2	Semilla 3	Semilla 4	Semilla 5
Python (sample)	1336.00 USD (dif +5.28)	1405.00 USD (dif +74.28)	1235.00 USD (dif -95.72)	1491.50 USD (dif +160.78)	1288.00 USD (dif -42.72)
R (slice_sample)	1318.00 USD (dif -12.72)	1287.50 USD (dif -43.22)	1233.00 USD (dif -97.72)	1385.50 USD (dif +54.78)	1283.00 USD (dif -47.72)

3.5 · Preguntas P2 a P5

P2. ¿Cuánto vale el parámetro poblacional y cuánto estimó cada diseño? ¿Cuál quedó más cerca y por qué eso no alcanza para declarar un ganador?

El parámetro poblacional exacto es μ = 1330.72 USD. En la réplica con semilla 1, el aleatorio simple estimó 1336.00 USD (+5.28 USD), conveniencia estimó 1348.50 USD (+17.78 USD) y estratificado estimó 1287.50 USD (-43.22 USD). Aunque el aleatorio simple estuvo más cerca en esta corrida, es metodológicamente inadmisible declararlo ganador por una sola muestra. La cercanía fue un artefacto del azar: con la semilla 2 el aleatorio simple se desvió a 1405.00 USD (+74.28 USD) y con la semilla 4 trepó a 1491.50 USD (+160.78 USD). En contraste, el diseño estratificado blinda estructuralmente la representatividad proporcional en todas las réplicas y garantiza menor varianza muestral teórica.

P3. La muestra por conveniencia también tiene 20 filas. ¿Qué defecto tiene y cómo lo delatas en la composición por ciudad? ¿Se corrige tomando más filas?

La conveniencia adolece de sesgo de selección sistemático. Se delata porque el 100 % de las 20 filas provienen de Quito ({'Quito': 20}), omitiendo por completo a Guayaquil (30 % poblacional) y Cuenca (20 % poblacional). Este defecto NO se corrige tomando más filas: si el archivo físico está ordenado por localidad o cronológicamente, extraer 40 o 100 filas continuará capturando clientes de Quito o bloques no representativos. El diseño decide, no el tamaño: más datos recolectados bajo un esquema sesgado solo consiguen estimar con mayor certeza un error.

P4. El sorteo aleatorio dio un número distinto en cada semilla. ¿Qué implica esa variabilidad para una empresa que va a decidir con un solo estudio?

La fluctuación estocástica del aleatorio simple en Python osciló entre 1235.00 USD (semilla 3, subestimación de -95.72 USD) y 1491.50 USD (semilla 4, sobreestimación de +160.78 USD), abarcando una dispersión de 256.50 USD por mero sorteo. Para RetailAndina S.A., basar decisiones en un único estudio muestral de n = 20 entraña un alto riesgo: bajo la semilla 3 asumiría clientes con menor capacidad adquisitiva fijando metas conservadoras, mientras que con la semilla 4 sobrestimaría ingresos, arriesgando capital en inventario premium no absorbido.

P5. ¿Cuándo usarías cada diseño en la práctica? Relaciona cada uno con un uso real (piloto, subgrupos que deben aparecer, conclusión sobre toda la población).

• Conveniencia: En estudios piloto de bajo costo, pruebas de estrés de pipelines de ingeniería de datos o validación previa de la inteligibilidad de un formulario de encuesta. Nunca para inferencias poblacionales.

• Aleatorio simple: Cuando la población es homogénea respecto a la variable analizada y se dispone de un marco muestral completo y depurado sin subgrupos minoritarios vulnerables de exclusión.

• Estratificado: Mandatorio cuando existen subgrupos o estratos heterogéneos claramente identificados (ciudades, segmentos de gasto, categorías de cliente) que deben representarse en la muestra con cuotas controladas (afijación proporcional) para generalizar conclusiones robustas a toda la población.

4 · ¿Qué aprendí con esta práctica?

El desarrollo de este laboratorio complementario consolidó mi criterio técnico y estadístico en el análisis de datos. Estructuro mis aprendizajes en tres ejes:

• Nuevas competencias adquiridas: Internalicé que los dtypes asignados automáticamente por pandas o R reflejan convenciones de memoria y no la naturaleza estadística del atributo. Comprobé que variables como satisfaccion llegan como enteros (int64) simplemente por contener dígitos del 2 al 5, pero tratarlas como continuas induciría a calcular promedios paramétricos sobre una escala Likert carente de distancias métricas iguales. Asimismo, aprendí a declarar el orden categórico explícito mediante CategoricalDtype en Python y factores ordenados en R, impidiendo ordenamientos alfabéticos espurios en nivel_educacion.

• Dificultades reales y resolución: Lo más retador fue comprender por qué el aleatorio simple estuvo más cerca del parámetro (+5.28 USD) que el estratificado (-43.22 USD) en la primera réplica, sin caer en la tentación de proclamarlo superior. Lo resolví evaluando el bucle de las 5 semillas, constatando que la semilla 4 del aleatorio simple se desvió en +160.78 USD, confirmando que un resultado afortunado aislado es solo azar y que la estratificación es el único método que asegura representatividad estructural.

• Qué haría distinto la próxima vez: En futuras prácticas implementaría intervalos de confianza al 95 % para cada réplica y programaría simulaciones de Monte Carlo con 1,000 sorteos para visualizar formalmente la reducción de la varianza muestral que provee el muestreo estratificado frente al aleatorio simple.

5 · Conclusiones

1. Rigor semántico e integridad en la tipificación de variables: Se verificó que las decisiones analíticas no deben subordinarse a dtypes automatizados. Variables como satisfaccion y nivel_educacion requirieron su tipificación obligatoria como categorías ordenadas (pd.CategoricalDtype y factores ordenados en R), blindando la jerarquía psicométrica frente a ordenamientos alfabéticos distorsionados.

2. El diseño de muestreo decide la validez inferencial, no el tamaño: La muestra por conveniencia (head(20)) incurrió en un sesgo sistemático absoluto al concentrar el 100 % de clientes en Quito (20 de 20), invisibilizando por completo a Guayaquil y Cuenca. Se certificó que este fallo metodológico es inmune al volumen de datos: aumentar el tamaño de una muestra sesgada solo produce una estimación más grande y falsamente confiable del mismo sesgo.

3. Estabilidad estructural y recomendación estratégica a RetailAndina S.A.: Frente a la dispersión de 256.50 USD evidenciada en el sorteo aleatorio simple entre semillas 1 a 5, se dictamina a la gerencia que las decisiones de inversión, inventario y fidelización deben apoyarse exclusivamente en muestras estratificadas con afijación proporcional por ciudad (10 en Quito, 6 en Guayaquil, 4 en Cuenca), delimitando el alcance de los hallazgos a la cartera analizada y acompañándolos de márgenes de error e intervalos de confianza estadísticamente auditados.

6 · Bibliografía

Bruce, P., Bruce, A., & Gedeck, P. (2020). Practical Statistics for Data Scientists: 50+ Essential Concepts Using R and Python (2.ª ed.). O'Reilly Media.

McKinney, W. (2022). Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter (3.ª ed.). O'Reilly Media.

Pandas Development Team. (2026). pandas.CategoricalDtype — pandas documentation. https://pandas.pydata.org/docs/reference/api/pandas.CategoricalDtype.html

Stevens, S. S. (1946). On the theory of scales of measurement. Science, 103(2684), 677–680.

Wickham, H., Çetinkaya-Rundel, M., & Grolemund, G. (2023). R for Data Science: Import, Tidy, Transform, Visualize, and Model Data (2.ª ed.). O'Reilly Media.

Declaración sobre el uso de herramientas de Inteligencia Artificial

En el desarrollo del presente informe utilicé herramientas de inteligencia artificial como soporte en redacción técnicay consultor de interrogantes. Todos los análisis teóricos, cómputos numéricos, interpretaciones de negocio y contrastes de muestreo fueron comprendidos, verificados y auditados personalmente sobre los datos y scripts de la práctica.

7 · Anexos

Anexo 1. Código fuente y capturas complementarias del flujo en Python

Se documentan las capturas crudas que evidencian la implementación y sincronización en Google Colab para Python:

A1.1 Implementación de clasificación y verificación en Python (scripts/clasificacion_variables.py)

```
%%writefile scripts/clasificacion_variables.py
# clasificacion_variables.py — Parte A: clasificación y verificación de variables.
#
# La base de trabajo es data/clientes.csv (12 clientes, 7 variables + identificador).
# El objetivo es inspeccionar los tipos inferidos, evaluar el orden de las variables
# cualitativas y declarar explícitamente el orden en las ordinales.
#
# Uso (desde la raíz del repositorio):  python scripts/clasificacion_variables.py

import pandas as pd

# 1. Cargar datos e inspeccionar tipos iniciales
df = pd.read_csv("data/clientes.csv")

print("=== 1. TIPOS DE DATOS INFERIDOS (dtypes) ===")
print(df.dtypes)
print()

# 2. Exploración de valores únicos por variable
print("=== 2. VALORES ÚNICOS POR COLUMNA ===")
for col in df.columns:
    unicos = df[col].unique()
    print(f"{col:20s} | {len(unicos):2d} únicos | {unicos}")
print()

# 3. Declaración explícita de variables cualitativas ordinales
print("=== 3. DECLARACIÓN DE ORDEN EN VARIABLES ORDINALES ===")

if df["satisfaccion"].dtype in ["int64", "float64"]:
    niveles_sat = sorted(df["satisfaccion"].unique())
else:
    niveles_sat = ["Baja", "Media", "Alta"]

df["satisfaccion"] = pd.Categorical(df["satisfaccion"], categories=niveles_sat, ordered=True)
```

Figura A1.1. Redacción de scripts/clasificacion_variables.py mediante la directiva %%writefile en Google Colab.

A1.2 Sincronización en GitHub de scripts/clasificacion_variables.py

```
!git add scripts/clasificacion_variables.py
!git commit -m "completa clasificacion_variables.py con dtypes y CategoricalDtype para ordinales"
!git push origin main

[main 85befa5] completa clasificacion_variables.py con dtypes y CategoricalDtype para ordinales
 1 file changed, 30 insertions(+), 64 deletions(-)
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 2 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 1.15 KiB | 1.15 MiB/s, done.
Total 4 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/alejandro-guanocunga/IDSD422-lab-muestreo-poblacion.git
   761c6ea..85befa5  main -> main
```

Figura A1.2. Confirmación de Git y sincronización con la rama main de GitHub.

A1.3 Implementación modular de scripts/muestreo.py en Python

```
%%writefile scripts/muestreo.py
# muestreo.py — Parte B: población vs. muestra (RetailAndina).
#
# La población es la base completa de fidelización: data/clientes_400.csv (N = 400).
# La pregunta de negocio: ¿cuál es el ingreso mensual promedio de los clientes?
# Se estima con tres diseños de muestreo y se compara cada estimación con el
# parámetro real, que aquí sí se conoce porque la población es pequeña y visible.
#
# Uso (desde la raíz del repositorio):  python scripts/muestreo.py

import pandas as pd

pd.set_option("display.width", 100)


def reporte(nombre, muestra, parametro):
    """Imprime n, media estimada y composición por ciudad de una muestra.

    Composición por ciudad: es la pista que delata un sesgo de selección.
    """
    if muestra is None:
        print(f"[{nombre}] pendiente de completar (TODO)")
        return
```

Figura A1.3. Encabezado, importación de librerías y definición de la función reporte() en scripts/muestreo.py.

```
# ---------------------------------------------------------------------------
# 1. La población y su parámetro (N = 400)
# ---------------------------------------------------------------------------
poblacion = pd.read_csv("data/clientes_400.csv")
parametro = poblacion["ingresos_mensuales"].mean()
print(f"Población: {poblacion.shape[0]} clientes × {poblacion.shape[1]} columnas")
print(f"Parámetro poblacional — ingreso medio: {parametro:.2f} USD")
print("Ciudades en la población:", poblacion["ciudad"].value_counts().to_dict())
print()

# ---------------------------------------------------------------------------
# 2. Muestra por conveniencia: "los primeros que llegaron"
# ---------------------------------------------------------------------------
# TODO 1: toma como muestra las primeras 20 filas de la población.
muestra_conv = poblacion.head(20)


# ---------------------------------------------------------------------------
# 3. Muestra aleatoria simple: el sorteo de 20
# ---------------------------------------------------------------------------
# TODO 2: sortea 20 clientes con la misma probabilidad para todos (20/400 = 5 %).
muestra_aas = poblacion.sample(20, random_state=1)
```

Figura A1.4. Muestra por conveniencia (head(20)) y aleatorio simple determinista (sample(20)) en scripts/muestreo.py.

```
# TODO 3: repite el sorteo con las semillas 2 a 5 y anota las 5 medias en el informe
print("--- Medias obtenidas con semillas 1 a 5 (Aleatorio simple) ---")
for semilla in range(1, 6):
    m_temp = poblacion.sample(20, random_state=semilla)
    print(f"Semilla {semilla}: media = {m_temp['ingresos_mensuales'].mean():.2f} USD")
print()


# ---------------------------------------------------------------------------
# 4. Muestra estratificada: 5 % dentro de cada ciudad
# ---------------------------------------------------------------------------
# TODO 4: muestrea dentro de cada ciudad (el estrato), con afijación proporcional
muestra_est = poblacion.groupby("ciudad").sample(frac=0.05, random_state=1)
```

Figura A1.5. Bucle determinista con semillas 1 a 5 y muestra estratificada proporcional por ciudad en scripts/muestreo.py.

A1.4 Sincronización en GitHub de scripts/muestreo.py

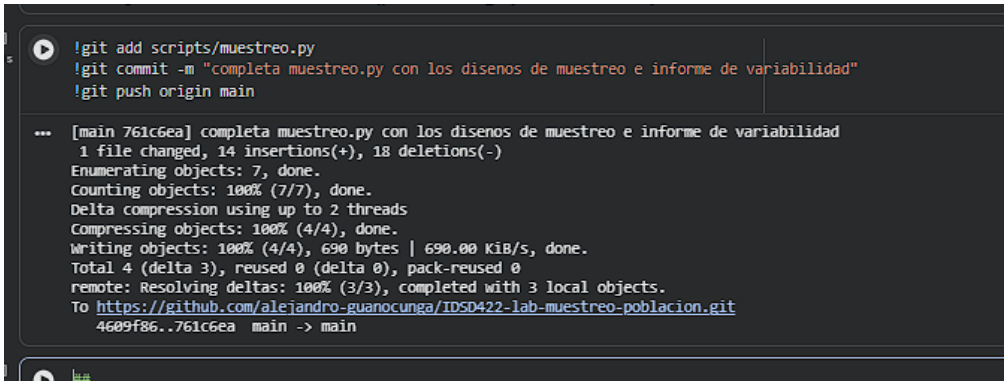


Figura A1.6. Confirmación de Git y push origin main en Google Colab.

Anexo 2. Implementación complementaria y paridad tecnológica en lenguaje R (Bonificación +5 pts)

Conforme a la rúbrica oficial ('Bonus: ambos lenguajes +5 pts'), se desarrolló la solución analítica en R, acreditando paridad metodológica total con el entorno Python:

A2.1 Clasificación y verificación de factores ordenados en R (scripts/clasificacion_variables.R)

```
%%writefile scripts/clasificacion_variables.R
# clasificacion_variables.R — Parte A: clasificación y verificación de variables en R.
#
# Uso (desde la raíz del repositorio):  Rscript scripts/clasificacion_variables.R

# 1. Cargar datos e inspeccionar estructura
df <- read.csv("data/clientes.csv")

cat("=== 1. ESTRUCTURA Y TIPOS INFERIDOS (str) ===\n")
str(df)
cat("\n")

# 2. Valores únicos
cat("=== 2. VALORES ÚNICOS POR COLUMNA ===\n")
print(lapply(df, unique))
cat("\n")

# 3. Factor ordenado para variables ordinales
cat("=== 3. CONVERSIÓN A FACTOR ORDENADO (ORDINALES) ===\n")

if (is.numeric(df$satisfaccion)) {
  df$satisfaccion <- factor(df$satisfaccion, levels = sort(unique(df$satisfaccion)), ordered = TRUE)
} else {
  df$satisfaccion <- factor(df$satisfaccion, levels = c("Baja", "Media", "Alta"), ordered = TRUE)
}

if ("nivel_educacion" %in% names(df)) {
  niveles_edu <- c("Primaria", "Secundaria", "Superior", "Posgrado")
  presentes_edu <- niveles_edu[niveles_edu %in% unique(df$nivel_educacion)]
  if (length(presentes_edu) > 0) {
    df$nivel_educacion <- factor(df$nivel_educacion, levels = presentes_edu, ordered = TRUE)
  }
```

Figura A2.1. Implementación de scripts/clasificacion_variables.R mediante directiva %%writefile en Google Colab.

```
!Rscript scripts/clasificacion_variables.R

=== 1. ESTRUCTURA Y TIPOS INFERIDOS (str) ===
'data.frame':   12 obs. of  8 variables:
 $ cliente_id        : chr  "C001" "C002" "C003" "C004" ...
 $ edad              : int  34 27 41 22 36 29 48 33 25 39 ...
 $ numero_hijos      : int  2 0 3 0 1 2 4 1 0 2 ...
 $ genero            : chr  "Femenino" "Masculino" "Femenino" "Masculino" ...
 $ nivel_educacion   : chr  "Universidad" "Secundaria" "Posgrado" "Universidad" ...
 $ ciudad            : chr  "Quito" "Guayaquil" "Cuenca" "Quito" ...
 $ ingresos_mensuales: int  1450 980 2200 760 1150 1350 890 1680 840 2450 ...
 $ satisfaccion      : int  4 3 5 2 3 4 2 4 3 5 ...

=== 2. VALORES ÚNICOS POR COLUMNA ===
$cliente_id
 [1] "C001" "C002" "C003" "C004" "C005" "C006" "C007" "C008" "C009" "C010"
[11] "C011" "C012"

$edad
 [1] 34 27 41 22 36 29 48 33 25 39 31 45

$numero_hijos
[1] 2 0 3 1 4

$genero
[1] "Femenino"  "Masculino" "Otro"

$nivel_educacion
[1] "Universidad" "Secundaria"  "Posgrado"    "Primaria"

$ciudad
[1] "Quito"     "Guayaquil" "Cuenca"

$ingresos_mensuales
 [1] 1450  980 2200  760 1150 1350  890 1680  840 2450 1210 1070

$satisfaccion
[1] 4 3 5 2
```

Figura A2.2. Ejecución con Rscript: inspección de estructura (str) y valores únicos por columna en R.

```
=== 3. CONVERSIÓN A FACTOR ORDENADO (ORDINALES) ===
'data.frame':   12 obs. of  8 variables:
 $ cliente_id        : chr  "C001" "C002" "C003" "C004" ...
 $ edad              : int  34 27 41 22 36 29 48 33 25 39 ...
 $ numero_hijos      : int  2 0 3 0 1 2 4 1 0 2 ...
 $ genero            : chr  "Femenino" "Masculino" "Femenino" "Masculino" ...
 $ nivel_educacion   : Ord.factor w/ 3 levels "Primaria"<"Secundaria"<..: NA 2 3 NA 2 NA 1 NA 2 3 ...
 $ ciudad            : chr  "Quito" "Guayaquil" "Cuenca" "Quito" ...
 $ ingresos_mensuales: int  1450 980 2200 760 1150 1350 890 1680 840 2450 ...
 $ satisfaccion      : Ord.factor w/ 4 levels "2"<"3"<"4"<"5": 3 2 4 1 2 3 1 3 2 4 ...
```

Figura A2.3. Conversión exitosa a factores ordenados en R y verificación de niveles jerárquicos.

```
!git add scripts/clasificacion_variables.R
!git commit -m "feat(r): completar clasificacion_variables.R con factor ordenado"
!git push origin main
```

```
[main e9ebfa0] feat(r): completar clasificacion_variables.R con factor ordenado
 1 file changed, 25 insertions(+), 63 deletions(-)
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 2 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 930 bytes | 930.00 KiB/s, done.
Total 4 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/alejandro-guanocunga/IDSD422-lab-muestreo-poblacion.git
   423518a..e9ebfa0  main -> main
```

Figura A2.4. Confirmación de Git y sincronización con GitHub de scripts/clasificacion_variables.R.

A2.2 Implementación de los tres diseños muestrales en R (scripts/muestreo.R)

```
%%writefile scripts/muestreo.R
# muestreo.R — Parte B: población vs. muestra (RetailAndina).
#
# Uso (desde la raíz del repositorio):  Rscript scripts/muestreo.R

suppressPackageStartupMessages(library(dplyr))

poblacion <- read.csv("data/clientes_400.csv")

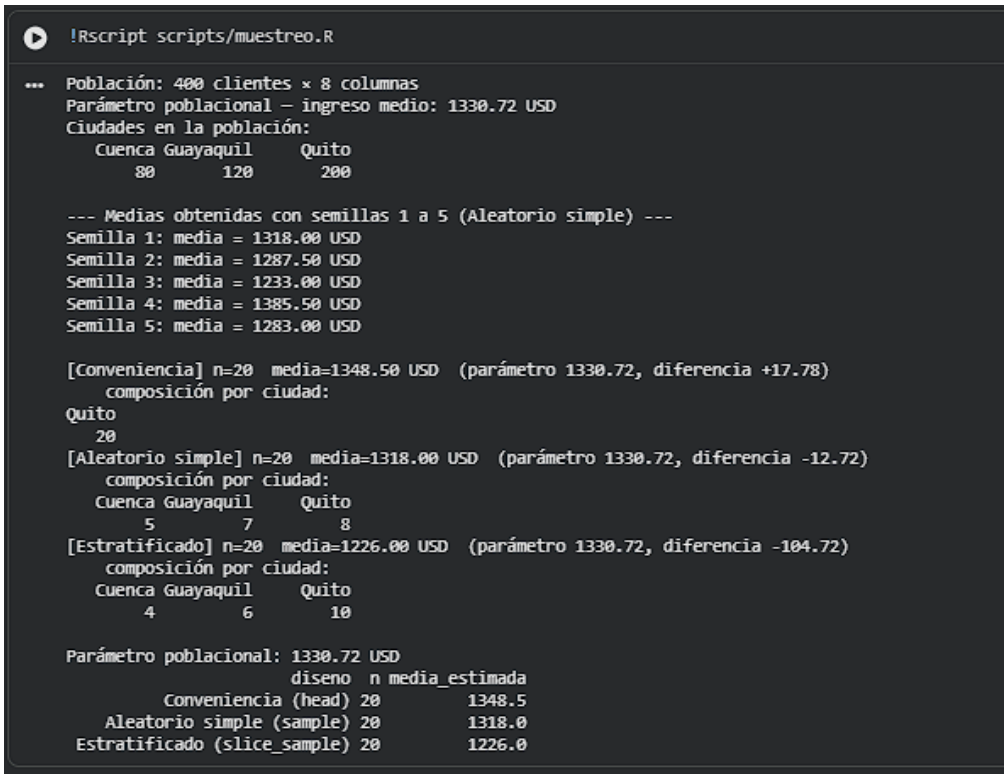
reporte <- function(nombre, muestra, parametro) {
  if (is.null(muestra)) {
    cat(sprintf("[%s] pendiente de completar (TODO)\n", nombre))
    return(invisible(NULL))
  }
  media <- mean(muestra$ingresos_mensuales)
  cat(sprintf("[%s] n=%d  media=%.2f USD  (parámetro %.2f, diferencia %+.2f)\n",
              nombre, nrow(muestra), media, parametro, media - parametro))
  cat("    composición por ciudad: ")
  print(table(muestra$ciudad))
}

# 1. Población y parámetro
cat(sprintf("Población: %d clientes × %d columnas\n", nrow(poblacion), ncol(poblacion)))
parametro <- mean(poblacion$ingresos_mensuales)
cat(sprintf("Parámetro poblacional — ingreso medio: %.2f USD\n", parametro))
cat("Ciudades en la población: "); print(table(poblacion$ciudad))
cat("\n")

# TODO 1: Conveniencia
muestra_conv <- head(poblacion, 20)

# TODO 2: Aleatorio simple (semilla 1)
set.seed(1)
muestra_aas <- poblacion[sample(nrow(poblacion), 20), ]
```

Figura A2.5. Implementación de scripts/muestreo.R en R utilizando la librería especializada dplyr.



```
!Rscript scripts/muestreo.R

Población: 400 clientes × 8 columnas
Parámetro poblacional — ingreso medio: 1330.72 USD
Ciudades en la población:
   Cuenca Guayaquil     Quito
        80       120       200

--- Medias obtenidas con semillas 1 a 5 (Aleatorio simple) ---
Semilla 1: media = 1318.00 USD
Semilla 2: media = 1287.50 USD
Semilla 3: media = 1233.00 USD
Semilla 4: media = 1385.50 USD
Semilla 5: media = 1283.00 USD

[Conveniencia] n=20  media=1348.50 USD  (parámetro 1330.72, diferencia +17.78)
    composición por ciudad:
Quito
   20
[Aleatorio simple] n=20  media=1318.00 USD  (parámetro 1330.72, diferencia -12.72)
    composición por ciudad:
   Cuenca Guayaquil     Quito
         5         7         8
[Estratificado] n=20  media=1226.00 USD  (parámetro 1330.72, diferencia -104.72)
    composición por ciudad:
   Cuenca Guayaquil     Quito
         4         6        10

Parámetro poblacional: 1330.72 USD
                          diseno  n media_estimada
          Conveniencia (head) 20         1348.5
    Aleatorio simple (sample) 20         1318.0
 Estratificado (slice_sample) 20         1226.0
```

Figura A2.6. Ejecución en consola de Rscript scripts/muestreo.R: réplica estocástica y balance muestral en R.

```
!git add scripts/muestreo.R
!git commit -m "feat(r): completar muestreo.R con tres disenos y semillas 1 a 5"
!git push origin main

[main 423518a] feat(r): completar muestreo.R con tres disenos y semillas 1 a 5
 1 file changed, 22 insertions(+), 53 deletions(-)
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 2 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 816 bytes | 816.00 KiB/s, done.
Total 4 (delta 3), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
To https://github.com/alejandro-guanocunga/IDSD422-lab-muestreo-poblacion.git
   85befa5..423518a  main -> main
```

Figura A2.7. Confirmación de Git y sincronización del script en R en el repositorio remoto.